productosDisponibles

InterfazGrafica

PanelAdmin

- panelCentral: JPanel
- cardLayout: CardLayout
- panelRegistrar: PanelRegistrarEmpleado
- panelCrearTorneo: PanelCrearTorneo
- panelGestionar: PanelGestionarTorneos
- panelVisualizaciones: PanelVisualizaciones
- panelInventarioAdmin: PanelInventarioAdmin
- panelMenuAdmin: PanelMenuAdmin
- panelTurnosAdmin: PanelTurnosAdmin
- btnCerrarSesion: JButton
- btnDashboard: JButton

- PanelAdmin()
- crearBotonNav(texto: String): JButton

PanelAlquilerEmpleado

- cafeteria: Cafeteria
- empleado: Empleado
- persistencia: GestorPersistencia
- tablaJuegos: JTable
- tablaAlquileres: JTable
- modeloJuegos: DefaultTableModel
- modeloAlquileres: DefaultTableModel
- btnAlquilar: JButton
- btnDevolver: JButton
- btnVolver: JButton
- lblEstado: JLabel

- PanelAlquilerEmpleado()
- seccion(texto: String): JLabel
- boton(texto: String): JButton
- tabla(modelo: DefaultTableModel): JTable
- configurarContexto(cafeteria: Cafeteria, empleado: Empleado, persistencia: GestorPersistencia): void
- setAccionVolver(accionVolver: Runnable): void
- refrescar(): void
- alquilarJuego(): void
- devolverJuego(): void
- buscarCopiaId(codCopia: String): CopiaPrestamo
- buscarPrestamoActivo(idCopia: String): Prestamo
- guardar(): void
- mostrarError(ex: Exception): void

PanelCajaMesero

- cafeteria: Cafeteria
- empleado: Empleado
- persistencia: GestorPersistencia
- comboProducto: JComboBox<ProductoOpcion>
- spinnerCantidad: JSpinner
- tablaCarrito: JTable
- modeloCarrito: DefaultTableModel
- btnAgregar: JButton
- btnRegistrarVenta: JButton
- btnVolver: JButton
- lblEstado: JLabel
- carrito: List<ItemVenta>

- PanelCajaMesero()
- etiqueta(texto: String): JLabel
- boton(texto: String): JButton
- configurarContexto(cafeteria: Cafeteria, empleado: Empleado, persistencia: GestorPersistencia): void
- setAccionVolver(accionVolver: Runnable): void
- refrescar(): void
- cargarProductos(): void
- agregarItem(): void
- registrarVentaCaja(): void
- descontarCopiasVendidas(): void

productosDisponibles

0..*

0..*

ProductoOpcion

- nombre: String
- producto: ProductoVendible
- precio: double
- copiaVenta: CopiaVenta

- ProductoOpcion(nombre: String, producto: ProductoVendible, precio: double, copiaVenta: CopiaVenta)
- toString(): String

PanelCliente

- layoutInterno: CardLayout
- contenedor: JPanel
- panelPrestamos: PanelPrestamosCliente
- panelCompras: PanelComprasCliente
- panelTorneos: PanelTorneosDisponibles
- panelFavoritos: PanelFavoritos
- panelPerfil: PanelMiPerfil
- accionCerrarSesion: Runnable

- PanelCliente()
- crearHome(): JPanel
- boton(texto: String): JButton
- configurarContexto(cafeteria: Cafeteria, cliente: Cliente, persistencia: GestorPersistencia): void
- setAccionCerrarSesion(accionCerrarSesion: Runnable): void

PanelComprasCliente

- cafeteria: Cafeteria
- cliente: Cliente
- persistencia: GestorPersistencia
- comboProducto: JComboBox<ProductoOpcion>
- spinnerCantidad: JSpinner
- txtCodigoDescuento: JTextField
- txtPuntosRedimir: JTextField
- tablaCarrito: JTable
- modeloCarrito: DefaultTableModel
- btnAgregar: JButton
- btnFinalizar: JButton
- btnVolver: JButton
- lblEstado: JLabel
- carrito: List<ItemVenta>
- productosDisponibles: List<ProductoOpcion>

- PanelComprasCliente()
- seccion(texto: String): JLabel
- etiqueta(texto: String): JLabel
- boton(texto: String): JButton
- tabla(modelo: DefaultTableModel): JTable
- configurarContexto(cafeteria: Cafeteria, cliente: Cliente, persistencia: GestorPersistencia): void
- setAccionVolver(accionVolver: Runnable): void
- refrescar(): void
- cargarProductos(): void
- agregarItem(): void
- finalizarCompra(): void
- parseEntero(texto: String): int
- descontarCopiasVendidas(): void

PanelComprasEmpleado

- cafeteria: Cafeteria
- empleado: Empleado
- persistencia: GestorPersistencia
- comboProducto: JComboBox<ProductoOpcion>
- spinnerCantidad: JSpinner
- txtCodigoDescuento: JTextField
- tablaCarrito: JTable
- modeloCarrito: DefaultTableModel
- btnAgregar: JButton
- btnFinalizar: JButton
- btnVolver: JButton
- lblEstado: JLabel
- carrito: List<ItemVenta>
- productosDisponibles: List<ProductoOpcion>

- PanelComprasEmpleado()
- seccion(texto: String): JLabel
- etiqueta(texto: String): JLabel
- boton(texto: String): JButton
- tabla(modelo: DefaultTableModel): JTable
- configurarContexto(cafeteria: Cafeteria, empleado: Empleado, persistencia: GestorPersistencia): void
- setAccionVolver(accionVolver: Runnable): void
- refrescar(): void
- cargarProductos(): void
- agregarItem(): void
- finalizarCompra(): void
- descontarCopiasVendidas(): void

PanelEmpleado

- layoutInterno: CardLayout
- contenedor: JPanel
- panelAlquiler: PanelAlquilerEmpleado
- panelCompras: PanelComprasEmpleado
- panelTorneos: PanelTorneosDisponibles
- panelTurnos: PanelTurnosEmpleado
- panelSugerir: PanelSugerirMenu
- panelFavoritos: PanelFavoritos
- panelCajaMesero: PanelCajaMesero
- panelEnsenarJuegos: PanelEnsenarJuegosDificiles
- empleadoActual: Empleado
- accionCerrarSesion: Runnable

- PanelEmpleado()
- crearHome(): JPanel
- boton(texto: String): JButton
- configurarContexto(cafeteria: Cafeteria, empleado: Empleado, persistencia: GestorPersistencia): void
- setAccionCerrarSesion(accionCerrarSesion: Runnable): void

PanelEnsenarJuegosDificiles

- cafeteria: Cafeteria
- empleado: Empleado
- persistencia: GestorPersistencia
- comboJuegos: JComboBox<Juego>
- btnRegistrarCapacitacion: JButton
- btnEnsenar: JButton
- btnVolver: JButton
- lblEstado: JLabel

- PanelEnsenarJuegosDificiles()
- boton(texto: String): JButton
- configurarContexto(cafeteria: Cafeteria, empleado: Empleado, persistencia: GestorPersistencia): void
- setAccionVolver(accionVolver: Runnable): void
- refrescar(): void
- registrarCapacitacion(): void
- ensenarReglas(): void

PanelFavoritos

- cafeteria: Cafeteria
- usuarioActual: Usuario
- persistencia: GestorPersistencia
- tablaCatalogo: JTable
- tablaFavoritos: JTable
- modeloCatalogo: DefaultTableModel
- modeloFavoritos: DefaultTableModel
- btnAgregar: JButton
- btnQuitar: JButton
- btnVolver: JButton

- PanelFavoritos()
- seccion(texto: String): JLabel
- boton(texto: String): JButton
- tabla(modelo: DefaultTableModel): JTable
- configurarContexto(cafeteria: Cafeteria, usuarioActual: Usuario, persistencia: GestorPersistencia): void
- setAccionVolver(accionVolver: Runnable): void
- refrescar(): void
- agregarFavorito(): void
- quitarFavorito(): void
- guardar(): void

PanelGestionarTorneos

- tablaTorneos: JTable
- modeloTabla: DefaultTableModel
- txtBuscar: JTextField
- btnBuscar: JButton
- btnFinalizar: JButton

- PanelGestionarTorneos()
- aplicarEstiloBoton(boton: JButton): void
- getBusqueda(): String
- getBtnBuscar(): JButton
- getBtnFinalizar(): JButton
- getModeloTabla(): DefaultTableModel
- getTablaTorneos(): JTable
- getIdTorneoSeleccionado(): String

Nota UML: "Todos los paneles del sistema heredan de JPanel (Swing).
Las relaciones de herencia con JPanel no se muestran individualmente para evitar sobrecargar visualmente el diagrama y mantener la legibilidad del UML."

Nota UML: "Los componentes JLabel utilizados en las interfaces no se representan individualmente en el diagrama para evitar saturación visual, ya que su uso es repetitivo y corresponde únicamente a elementos gráficos de Swing."

PanelTorneosDisponibles

- cafeteria: Cafeteria
- usuarioActual: Usuario
- persistencia: GestorPersistencia
- comboDia: JComboBox<DiaSemana>
- tablaTorneos: JTable
- modeloTabla: DefaultTableModel
- spinnerCupos: JSpinner
- btnActualizar: JButton
- btninscribirme: JButton
- btnDesinscribirme: JButton
- btnVolver: JButton
- lblEstado: JLabel

- + configurarContexto(cafeteria: Cafeteria, usuarioActual: Usuario, persistencia: GestorPersistencia): void
- + setAccionVolver(accion: Runnable): void
- + refrescarTabla(): void
- - inscribirUsuario(): void
- - desinscribirUsuario(): void
- - obtenerIdSeleccionado(): String
- - estaInscrito(torneo: Torneo, usuario: Usuario): boolean
- - guardarCambios(): void
- - configurarListeners(): void
- - ocultarColumnaId(): void

PanelInicio

PanelOpciones

- btnEmpleado: JButton
- btnAdministrador: JButton
- btnCliente: JButton

- getBtnEmpleado(): JButton
- getBtnAdministrador(): JButton
- getBtnCliente(): JButton

PanelOpcionesCliente

- Loggearme: JButton
- Registrarme: JButton

- getBtnLoggearse(): JButton
- getBtnRegistrarse(): JButton

PanelRegistrarEmpleado

- txtNombre: JTextField
- txtDocumento: JTextField
- txtLogin: JTextField
- txtPassword: JPasswordField
- txtCodigoDescuento: JTextField
- cbTipo: JComboBox<String>
- btnGuardar: JButton

- getNombre(): String
- getDocumento(): String
- getLoginEmpleado(): String
- getPasswordEmpleado(): String
- getCodigoDescuento(): String
- getTipoRol(): String
- getBtnGuardar(): JButton
- limpiarCampos(): void

PanelMiPerfil

- cafeteria: Cafeteria
- cliente: Cliente
- tablaPerfil: JTable
- tablaTorneo: JTable
- modeloPerfil: DefaultTableModel
- modeloTorneo: DefaultTableModel
- comboTorneos: JComboBox<String>
- btnVolver: JButton

- configurarContexto(cafeteria: Cafeteria, cliente: Cliente): void
- setAccionVolver(accion: Runnable): void
- refrescar(): void
- cargarTorneosDelCliente(): void
- refrescarTorneoSeleccionado(): void
- obtenerTipoCliente(): String
- buscarInscripcion(torneo: Torneo, usuario: Usuario): InscripcionTorneo

PanelLogin

- logBtton: JButton
- campoTextUsuario: JTextField
- campoContrasenia: JPasswordField

- getBotonLogin(): JButton
- getUsuario(): String
- getContrasenia(): String

PanelInventarioAdmin

- tablaJuegos: JTable
- modeloTabla: DefaultTableModel
- cbJuegos: JComboBox<String>
- cbCopiasPrestamo: JComboBox<String>
- cbCopiasVenta: JComboBox<String>
- txtCantidad: JTextField
- cbTipoCompra: JComboBox<String>
- btnActualizar: JButton
- btnComprar: JButton
- btnMoverVentaAPrestamo: JButton
- btnReparar: JButton
- btnMarcarRobado: JButton
- btnVerHistorial: JButton
- areaHistorial: JTextArea

- PanelInventarioAdmin()
- crearBoton(texto: String): JButton
- getTablaJuegos(): JTable
- getModeloTabla(): DefaultTableModel
- getCbJuegos(): JComboBox<String>
- getCbCopiasPrestamo(): JComboBox<String>
- getCbCopiasVenta(): JComboBox<String>
- getJuegoSeleccionado(): String
- getCopiaPrestamoSeleccionada(): String
- getCopiaVentaSeleccionada(): String
- getCantidad(): int
- getTipoCompra(): String
- getBtnActualizar(): JButton
- getBtnComprar(): JButton
- getBtnMoverVentaAPrestamo(): JButton
- getBtnReparar(): JButton
- getBtnMarcarRobado(): JButton
- getBtnVerHistorial(): JButton
- setTextoHistorial(texto: String): void
- limpiarCantidad(): void

PanelPrestamoCliente

- cafeteria: Cafeteria
- cliente: Cliente
- persistencia: GestorPersistencia
- spinnerPersonas: JSpinner
- comboNinos: JComboBox<String>
- comboJovenes: JComboBox<String>
- tablaJuegos: JTable
- tablaPrestamos: JTable
- modeloJuegos: DefaultTableModel
- modeloPrestamos: DefaultTableModel
- btnReservarMesa: JButton
- btnSolicitarPrestamo: JButton
- btnDevolver: JButton
- btnVolver: JButton
- lblEstado: JLabel

- configurarContexto(cafeteria: Cafeteria, cliente: Cliente, persistencia: GestorPersistencia): void
- setAccionVolver(accion: Runnable): void
- refrescar(): void
- reservarMesa(): void
- solicitarPrestamo(): void
- devolverPrestamo(): void
- buscarPrestamoActivo(idCopia: String): Prestamo
- contarCopiasDisponibles(juego: Juego): int
- guardar(): void
- mostrarError(ex: Exception): void

PanelGraficaBarras

- valoresCafeteria: double[]
- valoresJuegos: double[]
- etiquetasDias: String[]

- PanelGraficaBarras()
- actualizarDatos(cafeteria: double[]juegos: double[], etiquetas: String[]): void
- paintComponent(g: Graphics): void
- valorSeguro(arr: double[], i: int): double
- etiquetaSegura(i: int): String

PanelGraficaLineas

- reservasSemanales: int[]
- dias: String[]

- PanelGraficaLineas()
- actualizarDatos(reservasSemanales: int[]): void
- paintComponent(g: Graphics): void
- valorSeguro(i: int): int

PanelGraficaPastel

- copiasPrestamo: int
- copiasVenta: int
- nombresJuegos: String

- PanelGraficaPastel(prestamo: int, venta: int, nombresJuego: String)
- actualizarDatos(prestamo: int, venta: int, nombresJuego: String): void
- paintComponent(g: Graphics): void

"Los paneles de gráficas comparten dependencias con clases de renderizado gráfico de Java AWT/Swing (Graphics, Graphics2D, Color, Font, etc.).
Estas relaciones no se representan individualmente para mantener la claridad y legibilidad del diagrama UML."

PanelTurnosEmpleados

- cafeteria: Cafeteria
- empleado: Empleado
- persistencia: GestorPersistencia
- tablaTurnos: JTable
- modeloTabla: DefaultTableModel
- comboDiaCambio: JComboBox<DiaSemana>
- comboDiaIntercambio: JComboBox<DiaSemana>
- comboEmpleado: JComboBox<String>
- btnActualizar: JButton
- btnSolicitarCambio: JButton
- btnSolicitarIntercambio: JButton
- btnVolver: JButton
- lblEstado: JLabel

- configurarContexto(cafeteria: Cafeteria, empleado: Empleado, persistencia: GestorPersistencia): void
- setAccionVolver(accion: Runnable): void
- refrescar(): void
- solicitarCambio(): void
- solicitarIntercambio(): void
- buscarEmpleadoPorLogin(login: String): Empleado
- configurarListeners(): void

AdminController

- vista: VentanaPrincipal
- jefe: ControllerPrincipal
- panelAdmin: PanelAdmin
- sdfTabla: SimpleDateFormat

- cargarDatosIniciales(): void
- configurarListeners(): void

- configurarListenersPanelesFaltantes(): void
- registrarEmpleadoDesdeGUI(): void
- crearNuevoTorneo(): void
- cargarTorneosEnTablaFiltroDia: String): void
- finalizarTorneoSeleccionado(): void
- obtenerIdTorneoSeleccionado(): String
- configurarListenersInventario(): void
- refrescarInventario(): void

- cargarCopiasDelJuegoSeleccionado(): void
- comprarInventarioDesdeGUI(): void
- moverVentaAPrestamoDesdeGUI(): void
- reparar CopiaDesdeGUI(): void
- marcarRobadoDesdeGUI(): void
- verHistorialJuegoDesdeGUI(): void
- buscarCopiaPrestamo(juego: Juego, id: String): CopiaPrestamo
- buscarCopiaVenta(juego: Juego, id: String): CopiaVenta
- configurarListenersMenu(): void
- refrescarMenuYSugerencias(): void
- agregarProductoMenuDesdeGUI(): void
- aprobarSugerenciaDesdeGUI(): void
- rechazarSugerenciaDesdeGUI(): void
- configurarListenersTurnos(): void
- refrescarTurnosAdmin(): void
- construirTextoTurnos(empleado: Empleado): String
- asignarTurnoDesdeGUI(): void
- quitarTurnoDesdeGUI(): void
- aprobarSolicitudTurnoDesdeGUI(): void
- rechazarSolicitudTurnoDesdeGUI(): void
- validarAdminActual(): void
- guardarDatos(): void
- convertirDiaIdaCalendar(dia: String): int
- buscarJuegoPorNombre(nombreJuego: String): Juego
- buscarUsuarioPorLogin(login: String): Usuario
- buscarEmpleadoPorLogin(login: String): Empleado
- buscarCopiaVenta(juego: Juego, id: String): CopiaVenta

PanelRegistrarCliente

- logBtton: JButton
- campoTextUsuario: JTextField
- campoTextNombre: JTextField
- campoContrasenia: JPasswordField
- checkOpcion1: JCheckBox
- checkOpcion2: JCheckBox

- getBotonLogin(): JButton
- getUsuario(): String
- getNombre(): String
- getContrasenia(): String
- esMinio(): boolean
- esJoven(): boolean

PanelSugerirMenú

- cafeteria: Cafeteria
- empleado: Empleado
- persistencia: GestorPersistencia
- areaDescripcion: JTextArea
- lblEstado: JLabel
- btnEnviar: JButton
- btnVolver: JButton

- configurarContexto(cafeteria: Cafeteria, empleado: Empleado, persistencia: GestorPersistencia): void
- setAccionVolver(accion: Runnable): void
- enviarSugerencia(): void

PanelVisualizaciones

- cbJuegos: JComboBox<String>
- btnActualizar: JButton
- graficaPastel: PanelGraficaPastel
- graficaBarras: PanelGraficaBarras
- graficaLineas: PanelGraficaLineas
- cafeteria: Cafeteria

- configurarDatos(cafeteria: Cafeteria): void
- cargarComboJuegos(): void
- recalcularGraficas(): void
- recalcularGraficaPastel(): void
- recalcularGraficaBarras(): void
- recalcularGraficaLineas(): void
- buscarJuego(nombreJuego: String): Juego
- indiceDia(dia: DiaSemana): int

VentanaPrincipal

- CardLayout controlLayout
- JPanel contenedorPrincipal
- Runnable accionGuardarAlCerrar
- PanelOpciones
- PanelLogin
- PanelOpcionesCliente
- PanelRegistrarCliente
- PanelCliente
- PanelEmpleado
- PanelAdmin panelAdmin

+iniciarAplicacion()
+navegarA(String panelNombre)
-cargarDatos()
-guardarDatos()
-configurarComponentes(): void
-cambiarPantalla(): void
-guardarSYSalir(): void
+ main(args: String[]): void

Nota: "Por legibilidad del diagrama, no se representan individualmente las relaciones repetitivas con componentes Swing ni todas las herencias de paneles. Todos los Panel* heredan de JPanel y son gestionados por VentanaPrincipal."

PanelMenuAdmin

- tablaMenu: JTable
- tablaSugerencias: JTable
- modeloMenu: DefaultTableModel
- modeloSugerencias: DefaultTableModel
- txtNombreProducto: JTextField
- txtPrecioProducto: JTextField
- cbTipoProducto: JComboBox<String>
- txtAlergenos: JTextField
- btnActualizar: JButton
- btnAgregarProducto: JButton
- btnAprobarSugerencia: JButton
- btnRechazarSugerencia: JButton

- PanelMenuAdmin()
- crearBoton(texto: String): JButton
- limpiarFormularioProducto(): void

PanelTurnosAdmin

- tablaEmpleados: JTable
- tablaSolicitudes: JTable
- modeloEmpleados: DefaultTableModel
- modeloSolicitudes: DefaultTableModel
- cbEmpleado: JComboBox<String>
- cbDia: JComboBox<DiaSemana>
- btnActualizar: JButton
- btnAsignarTurno: JButton
- btnQuitarTurno: JButton
- btnAprobarSolicitud: JButton
- btnRechazarSolicitud: JButton

- PanelTurnosAdmin()
- crearBoton(texto: String): JButton

Controladores

AdminController

- vista: VentanaPrincipal
- jefe: ControllerPrincipal
- panelAdmin: PanelAdmin
- sdfTabla: SimpleDateFormat

+ AdminController(vista: VentanaPrincipal, jefe: ControllerPrincipal)
- cargarDatosIniciales(): void
- cargarJuegosEnCbTorneo(): void
- configurarListeners(): void
-
- configurarListenersPanelesFaltantes(): void
- registrarEmpleadoDesdeGUI(): void
- crearNuevoTorneo(): void
- cargarTorneosEnTablaFiltroDia: String): void
- finalizarTorneoSeleccionado(): void
- obtenerIdTorneoSeleccionado(): String
- configurarListenersInventario(): void
- refrescarInventario(): void

cargarCopiasDelJuegoSeleccionado(): void
- comprarInventarioDesdeGUI(): void
- moverVentaAPrestamoDesdeGUI(): void
- reparar CopiaDesdeGUI(): void
- marcarRobadoDesdeGUI(): void
- verHistorialJuegoDesdeGUI(): void
- buscarCopiaPrestamo(juego: Juego, id: String): CopiaPrestamo
- buscarCopiaVenta(juego: Juego, id: String): CopiaVenta
- configurarListenersMenu(): void
- refrescarMenuYSugerencias(): void
- agregarProductoMenuDesdeGUI(): void
- aprobarSugerenciaDesdeGUI(): void
- rechazarSugerenciaDesdeGUI(): void
- configurarListenersTurnos(): void
- refrescarTurnosAdmin(): void
- construirTextoTurnos(empleado: Empleado): String
- asignarTurnoDesdeGUI(): void
- quitarTurnoDesdeGUI(): void
- aprobarSolicitudTurnoDesdeGUI(): void
- rechazarSolicitudTurnoDesdeGUI(): void
- validarAdminActual(): void
- guardarDatos(): void
- convertirDiaIdaCalendar(dia: String): int
- buscarJuegoPorNombre(nombreJuego: String): Juego
- buscarUsuarioPorLogin(login: String): Usuario
- buscarEmpleadoPorLogin(login: String): Empleado

ControllerPrincipal

- vista: VentanaPrincipal
- controladorLogin: LoginController
- controladorOpcionesCliente: OpcionesClienteController
- controladorOpciones: OpcionesController
- controladorRegistrarCliente: ControladorRegistrarCliente
- controladorAdmin: AdminController
- esteCliente: Cliente
- usuarioActual: Usuario
- persistencia: GestorPersistencia

+ ControllerPrincipal(vista: VentanaPrincipal)
+ mostrarSeccion(nombrePantalla: String): void
- asegurarDatosMinimosParaGUI(): void
+ getCafeteria(): Cafeteria
+ getPersistencia(): GestorPersistencia
+ setUsuarioActual(usuario: Usuario): void
+ getUsuarioActual(): Usuario

ControladorRegistrarCliente

- vista: VentanaPrincipal
- jefe: ControllerPrincipal
- panelRegistro: PanelRegistrarCliente
- cafeteria: Cafeteria

+ ControladorRegistrarCliente(vista: VentanaPrincipal, jefe: ControllerPrincipal)
- configurarListeners(): void

OpcionesController

- vista: VentanaPrincipal
- jefe: ControllerPrincipal

+ OpcionesController(vista: VentanaPrincipal, jefe: ControllerPrincipal)
- configurarListeners(): void

LoginController

- vista: VentanaPrincipal
- jefe: ControllerPrincipal
- panelLogin: PanelLogin

+ LoginController(vista: VentanaPrincipal, jefe: ControllerPrincipal)
- configurarListeners(): void

OpcionesClienteController

- vista: VentanaPrincipal
- jefe: ControllerPrincipal
- panelOpcionesCliente: PanelOpcionesCliente

+ OpcionesClienteController(vista: VentanaPrincipal, jefe: ControllerPrincipal)
- configurarListeners(): void

ModuloVenta

«interface»
ProductoVendible

+ getTasaImpuesto(): double

itemsVenta

1..*

1

producto

ItemVenta

- cantidad: int
- precioUnitario: double

+ calcularImpuestoItem(): double
+ getSubtotalItem(): double

«abstract»
ProductoComestible

- nombre: String
- precioBase: double

0..*

CopiaVenta

- idUnico: String
- precioVenta: double

Pasteleria

- alergenos: List<String>

+ getAlergenos(): List<String>

Bebida

- esCaliente: Boolean
- esAlcoholica: Boolean

0..*

0..*

Venta

- idVenta: int
- fecha: Date
- subtotal: double
- impuestos: double
- propinaValor: double
- total: double
-descuentoAplicado: int
-puntosGenerados: int
- realizadaPor: Usuario

+ calcularSubtotal(): double
+ calcularImpuestosTotales(): double
+ calcularPropina(): double
+ setPropina(porcentaje: double): void
+ calcularTotal(): double
+ calcularPuntosGenerados(): int
+ aplicarDescuento(codigo: String): void
+ aplicarBono(bono: BonoTorneoAmistoso): void
+ setTotal(total: double): void

0..1

inventarioVentas

0..*

GestorVentas

-gestorPersistencia: gestorpersistencia.
-ventas: List<Venta>

+ registrarVenta(venta: Venta): void
+ getVentas(inicio: Date, fin: Date): List<Venta> +
generarInformeVentas(inicio: Date, fin: Date, granularidad: String): String

0..*

ModuloVenta

ConsolaEmpleado

+ main(args: String[]): void
# mostrarMenu(): void
# ejecutarOpcion (opcion: int): void
- menuInscribirTorneo(): void

ConsolaCliente

+ main(args: String[]): void
# hacerLogin(): boolean
- registrarCliente(): void
mostrarMenu(): void
ejecutarOpcion (opcion: int): void
- menuInscribirTorneo(): void
- menuDesinscribirTorneo(): void

«abstract»
Consola

- scanner: Scanner

+ iniciarAplicacion(): void
-cargar Datos(): void
-guardarDatos(): void
hacerLogin(): boolean
pedirEntero (mensaje: String): int
pedirCadena (mensaje: String): String
#mostrarMenu():void
#ejecutarOpcion(opcion): void

ConsolaAdministrador

+ main(args: String[]): void
# mostrarMenu(): void
# ejecutarOpcion (opcion: int): void
- menuCrear Torneo(): void
- menuRegistrar Empleado(): void

Persistencia

GestorPersistencia

- rutaArchivos: String

+ cargarTodo(): Cafeteria
+ guardarTodo(c: Cafeteria): void
+ cargarUsuarios(): List<Usuario>
+ guardarUsuarios(lista: List<Usuario>): void
+ cargarJuegos(): List<Juego>
+ guardarJuegos(lista: List<Juego>): void
+ cargarVentas(): List<Venta>
+ guardarVentas(lista: List<Venta>): void
+ cargarPrestamos(): List<Prestamo>
+ guardarPrestamos(lista: List<Prestamo>): void
+ cargarTorneos(juegosTotales: List<Juego>): List<Torneo>
+ guardarTorneos(torneos: List<Torneo>): void
+ cargarInscripcionesTorneos(gestor: GestorTorneo, usuariosTotales: List<Usuario>): void
+guardarInscripcionesTorneos(torneos: List<Torneo>): void
+ cargarPagosTorneos(gestor: GestorTorneo): void
+ guardarPagosTorneos(torneos: List<Torneo>): void
+ cargarBonosTorneos(gestor: GestorTorneo, usuariosTotales: List<Usuario>): void
+ guardarBonosTorneos(bonos: List<BonoTorneoAmistoso>): void
+ cargarGestorTorneos(usuariosTotales: List<Usuario>, juegosTotales: List<Juego>): GestorTorneo
+ guardarGestorTorneo(gestorTorneo: GestorTorneo): void

Juego

- nombre: String
- anioPublicacion: Int
- empresaMatriz: String
- minJugadores: Int
- maxJugadores: Int
- edadMinima: Int
- categoria: String
- esDificil: Boolean
- copiasParaPrestamo: List<CopiaPrestamo>
- copiasParaVenta: List<CopiaVenta>

+ getCopiaDisponible(): CopiaPrestamo
+ getCopiasPrestamo(): List<CopiaPrestamo>
+ getCategoria(): String
+ esAptoParaEdad(edadMinimaEnMesa: int): boolean
+ soportaNPersonas(n: int): boolean
+ estaDisponibleParaVenta(): boolean
+ agregarCopiaPrestamo(CopiaPrestamo copia): void
+ agregarCopiaVenta(CopiaVenta copia): void

Cafeteria

- capacidadMax: int
- cantPersonasActuales: int
- nombreEstablecimiento: String
-List<Juego> juegos
-List<Mesa> mesas
-List<Prestamo> historialPrestamos
-GestorUsuarios gestorUsuarios
-GestorVentas gestorVentas

+ getInstance(): Cafeteria
+ login(login: String, pass: String): Usuario
+ hayCapacidad(nPersonas: int): boolean
+ registrarIngreso(nPersonas: int): boolean
+ registrarSalida(nPersonas: int): void
+ buscarJuego(nombre: String): Juego
+ getMesaDisponible(personas: int): Mesa
+ validarMinimoEmpleados(dia: DiaSemana): boolean
+ consultarAlergenos(p: Pasteleria): List<String>

CopiaPrestamo

- idUnico: String
- estado: String
- estaDisponible: Boolean
- vecesPrestado: int

+ prestar(): void
+ devolver(): void
+ marcarDesaparecida(): void
+ reparar(): void
+ estaDisponible(): boolean
+ incrementarContadorPrestamos(): void

1..2

Mesa

- idMesa: Int
- capacidadMax: Int
- ocupada: Boolean
- cantPersonasActuales: Int
- hayNinos: Boolean
- hayJovenes: Boolean
- hayBebidaCaliente: Boolean

+ ocupar(personas: int, hayNinos: boolean, hayJovenes: boolean): void
+ liberar(): void
+ estaDisponible(): boolean
+ puedeRecibirBebidaCaliente(): boolean
+ puedeRecibirBebidaAlcoholica(): boolean
+ puedeRecibirJuegoAccion(): boolean

0..3

Préstamo

-estado: String
-fechaHoralnicio: Datetime
-fechaHoraFin: Datetime

+ finalizar(): void
+ estaVencido(): boolean
+ getDuracionHoras(): double

Usuario

GestorUsuarios

+ registrarCliente(login: String, password: String, nombre: String, esNino: boolean, esJoven: boolean): Cliente
+ registrarEmpleado(login: String, password: String, nombre: String, tipo: String, codigoDescuento: String): Empleado
+ registrarAdministrador(login: String, password: String, nombre: String): Administrador
+ autenticar(login: String, password: String): Usuario
+ buscarUsuario(login: String): Usuario
+ existeUsuario(login: String): boolean
+ eliminarUsuario(login: String): boolean

users

«abstract» Usuario

- login: String
- password: String
- nombre: String

+ iniciarSesion(login: String, pass: String): boolean
+ cerrarSesion(): void
+ cambiarPassword(nuevoPass: String): boolean
+ agregarFavorito(j: Juego): void
+ eliminarFavorito(j: Juego): void

Administrador

+ aprobarSolicitudTurno(s: SolicitudTurno): void
+ rechazarSolicitudTurno(s: SolicitudTurno): void
+ comprarJuego(j: Juego, cant: int, tipo: String): void
+ darJuegoPorRobado(c: CopiaPrestamo): void
+ reparar Juego(c: CopiaPrestamo): void
+ moverJuegoAVenta(c: CopiaPrestamo): void
+ generarInformeVentas(ini: LocalDateTime, fin: LocalDateTime, granularidad: String): GestorVentas(delegado)
+ aprobarSugerenciaMenu(s: SugerenciaMenu): void
+ agregarProductoMenu(p: ProductoComestible): void
+ verHistorialJuego(j: Juego): String
+ registrarEmpleado(login: String, password: String, nombre: String, tipo: String, codigoDescuento: String): Empleado
- puedeAusentarse(solicitante: Empleado, dia: DiaSemana, cafe: Cafeteria): bool
+ asignarTurno(emp: Empleado, dia: DiaSemana): void
+ quitarTurno(emp: Empleado, dia: DiaSemana, cafe: Cafeteria): void
+ rechazarSugerenciaMenu(s: SugerenciaMenu): void
+ registrarEmpleado(login: String, password: String, nombre: String, tipo: String, codigoDescuento: String): Empleado

<<Enum>> DiaSemana

Lunes,Martes,Miércoles,Jueves, Viernes, Sábado, Domingo

Cliente

- esNino: boolean
- esJoven: boolean
- juegosReservados: int
- puntosFidelidad: int
- mesaActual: Mesa

+ reservarMesa(personas: int, hayNinos: boolean, hayJovenes: boolean): Mesa
+ solicitarPrestamo(copia: CopiaPrestamo, mesa: Mesa): Prestamo
+ devolverJuego(p: Prestamo): void
+ realizarCompra(items: List<ItemVenta>): Venta
+ usarPuntosFidelidad(puntos: int): double
+ liberarMesa(): void
+ marcarFavorito(j: Juego): void
+ desmarcarFavorito(j: Juego): void
+ tieneFavorito(j: Juego): void
+ realizarCompra(items: ItemVenta[], codigoDescuento: String, puntosARedimir: int): Venta
- validarCodigoEmpleado(String: código): bool

«abstract» Empleado

- codigoDescuento: String

+ solicitarCambioTurno(dia: DiaSemana): SolicitudTurno
+ solicitarIntercambioTurno(otro: Empleado, dia: String): SolicitudTurno
+ sugerirPlato(desc: String): SugerenciaMenu
+ realizarCompra(items: List<ItemVenta>): Venta
+ alquilarJuego(copia: CopiaPrestamo): Prestamo
+ estaEnTurno(): boolean
+ consultarDiaAsignado(): List<DiaTurno>
- obtenerDiaHoy(): DiaSemana

SugerenciaMenu

- descripcion: String
- estado: String

SolicitudTurno

- descripcion: String
- estado: String
- esIntercambio: boolean

Semana

- dias: Map<DiaSemana, DiaTurno>

+ getTurnoDelDia(dia: DiaSemana): DiaTurno
+ actualizarTurno(dia: DiaSemana, nuevoTurno: DiaTurno): void

DiaTurno

- dia: DiaSemana
- estaAprobado: boolean

+ getDia(): DiaSemana
+ estaAsignado(): boolean
+ asignar(): void
+ liberar(): void
+ isAprobado(): boolean
+ setAprobado(estado: boolean): void

Cocinero

Mesero

+ registrarVenta(items: List<ItemVenta>, mesa: Mesa): Venta
+ puedeEntrenar(j: Juego): boolean
+ agregarJuegoConocido(j: Juego): void
+ registrarVenta(items: ItemVenta[], mesa: Mesa): Venta

Torneos

GestorTorneos

+ crearTorneoAmistoso (admin: Administrador, juego: Juego, dia: DiaSemana, hora: String, cupoTotal: int, valorBono: double): TorneoAmistoso
+ crear TorneoCompetitivo (admin: Administrador, juego: Juego, dia: DiaSemana, hora: String, cupoTotal: int, tarifa: double): TorneoCompetitivo
+ getTorneos (dia: DiaSemana): List<Torneo>
+ inscribir (usuario: Usuario, torneoId: String, cantidad Cupos: int): void
+ desinscribir (usuario: Usuario, torneoId: String): void
+ finalizarTorneo (admin: Administrador, torneoId: String, ganador: Usuario): Resultado Torneo
+ calcularCupos Reservados (cupoTotal: int): int
+ validarCupoVsCopias Prestamo (juego: Juego, cupoTotal: int): boolean
+ validarEmpleadoSinTurno (empleado: Empleado, dia: DiaSemana, hora: String): boolean

<<Enum>> EstadoTorneo

Programado, En_Curso, Finalizado, Cancelado

ResultadoTorneo

- premioMetálico: double

«abstract» Torneo

- idTorneo: String
- dia: DiaSemana
- fechaInicio: LocalTime
- duracionMin: int
- estado: EstadoTorneo
- cupoTotal: int
- cupoReservadoFanaticos: int
- cupoOcupadoReservado:int
- cupoOcupadoRegular: int
- fechaCreacion: Date
- reglaMaxCuposPorUsuario: int = 3 (const)
- nombre: String

+ cuposDisponiblesReservados(): int
+ cuposDisponiblesRegulares(): int
+ esFanatico(u: Usuario): boolean
+ validarCupoMaximoPorUsuario (cantidad: int): boolean
+ puedeInscribirse (u: Usuario, cantidad: int): boolean

TorneoAmistoso

- valorBono: double
- bonoOtorgado: boolean

+ otorgarBono(ganador: Usuario): BonoDescubrimiento

Nota UML: los cupos reservados deben calcularse como ceil(0.2*cupoTotal) (redondeo hacia arriba).

TorneoCompetitivo

- tarifaEntrada: double
- pozoPremio: double
- porcentajePozoPremio: double

+ calcularPozo(): double
+ esElegiblePremioMetalico (u: Usuario): boolean
+ registrarPago (u: Usuario, monto: double): void

esElegiblePremioMetalico -> ? (false si empleado) registrarPago -> (aunque el pago sea tercerizado, guardas el registro)

InscripcionTorneo

- id_inscripcion: String
- fecha: Date
- cantidadCupos: int
- cuposReservadosUsados: int
- cuposRegularesUs